

Lab assignment 3: Language similarity

Introduction to programming (LIS020)

Robert Östling
robert@ling.su.se

Amanda Kann
amanda.kann@ling.su.se

July 10, 2026

Purpose and learning outcomes

In this assignment, you will produce code to compute the lexical similarity between languages using word lists, which is important in historical linguistics and can be used to infer the genealogical relationships between languages. You will also become familiar with using existing unit tests.

To achieve a given grade for this assignment, you will need to:

- **Pass:** Understand and implement the *Levenshtein algorithm*, a method to compute the *edit distance* between two strings. This is done by modifying an existing partially completed program, following detailed instructions in this assignment.

During the oral check-in, you should be able to demonstrate and explain in broad strokes how your implementation works, and show that your code passes all *Pass*-level unit tests.

- **Pass with distinction:** Extend the Levenshtein algorithm to differentiate between separate categories of differences, for instance so that a differing vowel counts as less different than a differing consonant. This involves independently modifying existing code with less detailed instructions.

During the oral check-in, you should also be able to demonstrate and explain your extension, and show that your code passes all *Distinction*-level unit tests.

Submitting your solution

Submit the file `similarity.py` in Canvas, and sign up for an oral check-in time slot in the Canvas calendar.

Background

The *Levenshtein algorithm* (or *minimal edit distance algorithm*) computes the number of edit operations needed to transform one string into another. The operations normally allowed are insertion, deletion and replacement. For instance, the word ‘hello’ can be transformed into ‘shall’ by inserting an *s*, deleting the *o*, and replacing the *e* with an *a*. There are also several other ways of changing ‘hello’ into ‘shall’, but you can never do it in less than the three edits suggested above, so the *Levenshtein distance* between ‘hello’ and ‘shall’ is three. Here, we denote this as $D(\text{hello}, \text{shall}) = 3$.

Jurafsky and Martin (2024, section 2.8) contains a detailed description of how to compute the Levenshtein distance between two strings, and pseudo-code is given in their Figure 2.17. The Levenshtein algorithm is frequently used in language technology, and is an example of an even more general and useful class of algorithms called *dynamic programming*, so make an effort to understand how it works. We will discuss all this in even more depth during the next programming course, Applied Programming (LIS050).

The Automated Similarity Judgment Program (ASJP) database (Holman et al., 2011; Wichmann et al., 2022) contains short word lists for 5,590 different languages, using a simplified phonetic notation called ASJPcode. For instance, the word for the concepts FISH and TREE in a number of languages is given as:

Language	FISH	TREE
English	fiS	tri
Swedish	fisk	trEd
Italian	peSe	albero
Spanish	peskado, pes	arbol, palo

Note that in some cases, more than one form may be given, as in the Spanish example above. By using the Levenshtein algorithm we can now see how much two languages differ lexically with respect to these two concepts. For instance, English and Swedish differ relatively little:

$$D(\text{fiS}, \text{fisk}) + D(\text{tri}, \text{trEd}) = 2 + 2 = 4$$

whereas English and Italian differ more:

$$D(\text{fiS}, \text{peSe}) + D(\text{tri}, \text{albero}) = 3 + 5 = 8$$

In order to account for the different length of words, we can define the *normalized Levenshtein distance* (NLD) between two strings to be the Levenshtein distance between the strings, divided by the length of the longest of the two strings. This is always a value between 0 and 1. For instance:

$$\text{NLD}(\text{fiS}, \text{fisk}) = D(\text{fiS}, \text{fisk})/4 = 2/4 = 0.5$$

Tasks

Start by giving the following command to copy the lab template and tests to your home directory:

```
cd ~/lis020
cp -r /home/dsv/robe/lis020/labs/similarity lab3
```

You should edit the file `similarity.py` to achieve the goals below.

1. There is an empty function called `levenshtein_distance`. Guided by the the pseudo-code from (Jurafsky and Martin, 2024, Figure 2.17), fill in the contents of this function, and make it return the Levenshtein distance between its two string arguments. You can test it by calling the function with the two strings ('intention' and 'execution') whose distance matrix is given in (Jurafsky and Martin, 2024, Figure 2.18), to verify that your code produces the same intermediate values (but please check the hints below for an important detail!)

Hints:

- You can create an all-zero matrix easily by using a list comprehension and the `range` function with argument `m` to create `m` rows. Each row can be created with `[0]*n` where `n` is the number of columns.
 - Jurafsky and Martin use substitution weight 2 in their example in Figure 2.18, whereas we assume a substitution weight of 1. You can simply change this weight to 2 while testing, if you want to use Figure 2.18 as a reference. Don't forget to change it back to 1 when you are done!
2. There is an empty function called `concept_nld`, which takes two lists of word forms. Make it return the *mean normalized Levenshtein distance* (mean NLD) between the two lists. This requires computing the NLD of each pair. For instance, this function can be used for comparing the words for FISH in Italian and Spanish:

```
concept_nld(['peSe'], ['peskado', 'pes'])
```

The value of this should be $17/28 \approx 0.6071$, which is the mean of $NLD(\text{peSe}, \text{peskado}) = 5/7 \approx 0.7143$ and $NLD(\text{peSe}, \text{pes}) = 2/4 = 0.5$

3. **For distinction:** Modify the function `levenshtein_distance` so that vowel changes may count as a smaller distance than other types of edits. Formally, this is a type of *weighted* Levenshtein distance, where all edits have the weight 1.0 except substitution of one vowel for another

vowel, which have another weight. For instance, if the vowel change weight is 0.5, after this modification the distance between ‘tri’ and ‘trEd’ should be 1.5, since there is one vowel change (‘i’ to ‘E’) and one consonant insertion (‘d’). The following symbols are used for vowel sounds in ASJPcode: 3 a e E i o u. Your extended function should be backwards compatible with the original `levenshtein_distance` function, by passing an *optional argument* called `vowel_weight`, whose default value should be 1.0. For instance, the return value of the following should be 1.5:

```
levenshtein_distance("tri", "trEd", vowel_weight=0.5)
```

whereas if you leave out `vowel_weight`, the standard Levenshtein algorithm should be used, and the return value becomes 2.0:

```
levenshtein_distance("tri", "trEd")
```

Unit tests

Execute the following command to run our provided *unit tests* for **passing**:

```
python3 -m unittest tests/pass/*.py
```

This will run Python’s built-in `unittest` module, which loads our code that tests this particular lab assignment. If everything works, you will see some text ending with an ‘OK’. If something is wrong, you will instead see information about what we expected would happen, and what actually happened. For instance, if you forgot to add a distance for deletions in the Levenshtein algorithm, you will see a number of messages like this:

```
FAIL: test_levenshtein_delete_last
(tests.pass.test_similarity.LevenshteinTestCase.test_levenshtein_delete_last)
-----
Traceback (most recent call last):
  File ".../language-similarity/tests/pass/test_similarity.py",
    line 69, in test_levenshtein_delete_last
    self.assertAlmostEqual(
AssertionError: 0.0 != 1.0 within 7 places (1.0 difference) :
incorrect Levenshtein distance for abc -> ab
```

This tells you that when the unit test checked what `levenshtein_distance("abc", "ab")` returns back, it expected the value 1.0 (because the ‘c’ at the end was deleted), but in fact it got the value 0.0. There will also be some additional errors because the unit tests also use the `levenshtein_distance` function to compute language distances, which will of course all be wrong if the Levenshtein algorithm is not correctly implemented.

If you aim for **pass with distinction**, you also need to pass the following tests, *in addition* to the above:

```
python3 -m unittest tests/distinction/*.py
```

This simply performs a few more tests, with different vowel weights.

References

- Holman, E., Brown, C., Wichmann, S., Müller, A., Velupillai, V., Hammarström, H., Sauppe, S., Jung, H., Bakker, D., Brown, P., Belyaev, O., Urban, M., Mailhammer, R., List, J.-M., and Egorov, D. (2011). Automated dating of the world’s language families based on lexical similarity. *Current Anthropology*, 52:841–875.
- Jurafsky, D. and Martin, J. H. (2024). *Speech and Language Processing*. 3rd edition (draft), available online at <https://web.stanford.edu/~jurafsky/slp3/>.
- Wichmann, S., Holman, E. W., and Brown, C. H. (2022). The ASJP database (version 20). <https://asjp.clld.org/>.